

C++基础：基本数据类型 与导出数据类型

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

基础数据类型

- 写程序主要是对数据进行计算或处理，也就是使用 C++ 语言支持的数据类型和运算，完成计算任务。
- 如下的两个程序分别列出来两种不同的计算程序，分别涉及了计算符号和数据类型的定义。

course1 >  sum_test.cpp > ...

```
1 //sum_test.cpp
2 #include<iostream>
3 using namespace std;
4 int main()
5 {
6     int a,b,sum;
7     a = 43;
8     b = 37;
9     sum = a + b;
10    cout << "The sum is " << sum;
11    cout << endl;
12    return 0;
13 }
```

course1 >  area_of_cicle.cpp > ...

```
1 //program 3-2.cpp
2 #include<iostream>
3 using namespace std;
4 int main()
5 {
6     const float pai = 3.14;
7     float radius;
8     cout << " Enter radius:";
9     cin >> radius;
10    float area = pai * radius * radius;
11    cout << "\n The area of circle is ";
12    cout << area << endl;
13    return 0;
14 }
```

基础数据类型

- 类型的信息决定了变量值的表示方法以及所需的内存量，类型的信息还决定了相关的算术运算的意义。
- 类型概念的几个要点是：
 - 每一项数据应唯一地属于某种类型。
 - 每一数据类型意味着一个有明确定义的值的集合。
 - 同一类型的数据占用相同大小的存储空间。
 - 同一类型的数据具有相同的（允许对其施加的）运算操作集。
- C++程序中的数据类型可以分为如下几类：

表 3.1 C++语言中类型的划分

系统提供	基本类型	int、float、double、char、bool、void
	派生类型	(修饰符+基本类型)
用户定义	完全由用户定义	class, (struct, union)
	部分由用户定义	enum (int 类型的子集)
由其他类型导出		array、struct、pointer、reference

基本数据类型

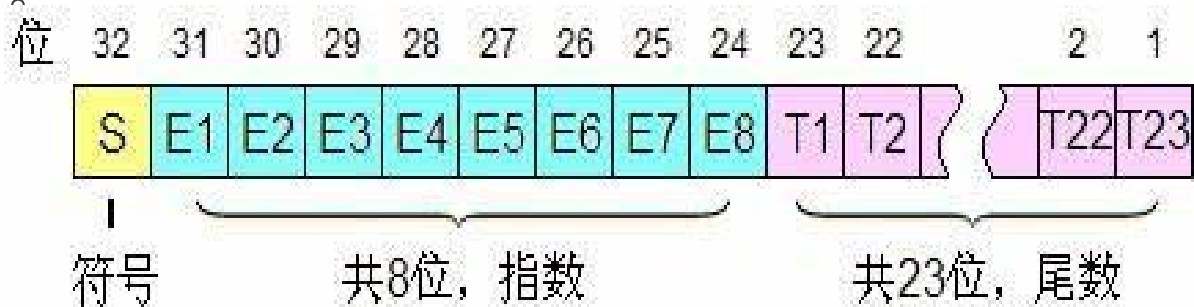
- 基本类型是具有下面 3 个特征的数据类型：
 - 由系统定义和提供。
 - 它们是构造所有其他类型的原始出发点。
 - 它们是几乎所有程序设计语言都包含的数据类型。
- C++语言的基本数据类型有：int 型、float 型、double 型、char 型、bool型和 void 型。

基本数据类型

- **int** 型又称整型。是最常用最基本的数据类型。
- **范围：**原则上是所有整数，实际的值集为计算机所能表示的所有整数。这个范围是有限的，因此程序员在编程时必须经常考虑因数据过大而溢出的问题。
- **存储：**占用的存储空间按不同的计算机和编译系统而有所差别，目前在PC机上运行的各种 C++语言规定 **int**型数据占用 4 B即 32 bit 空间。
- **运算：**int 型数据允许算术运算、关系运算等许多种运算。

基本数据类型

- **float** 型又称浮点型，它对应着数学中的实数概念，带小数点的数。
- **float** 型的值集，原则上是任意大小和精度的小数，实际的值集虽然不可能是任意大小，但由于采用尾数+阶码的表示形式，所以其可表示浮点数的范围可大到 $\pm 3.4 \times 10^{38}$ ，表示的精度可以小到 1.0×10^{-38} 。因此，在一般的应用问题中，**float** 型数据总是可以满足精度和大小的要求，不会出现溢出现象。
- **float** 型数据一般占用 **4 B**，即 **32 bit** 空间。当精度较高或数值较大时，人们往往使用 **double** 型（占用 **8 B**，即 **64 bit** 空间）。
- **float** 型数据与 **int** 型数据的区别在于它们所参加的运算操作类型是不同的。比如 **int** 型数据常进行增减量运算（++，--），条件判断等语句中。



基本数据类型

- `char` 型又称字符型，即把单个字符作为一种数据处理。
- `char` 型的值集是全部基本字符、ASCII 码集或扩充的 ASCII 码集对应的全部符号。
- `char` 型的数据占用 **1 B**即 **8 bit** 空间。
- 在作为数字计算的时候，`char` 型数据与低位宽的 `int` 型等价，因此可参加的运算相当广泛。
- 字符集可与单字节整数有完整的对应关系（ASCII 码），因此还可把 `char` 型看做是可以用来表示单字节整数的字符型。

基本数据类型

- `bool` 型在新版的 C++ 语言中被列为基本类型，它只有两个值：`false`、`true`，表示逻辑的真和假，可参加逻辑运算和作为逻辑表达式及关系表达式的结果。`bool` 型的存储空间为 **1B**，即 **8bit**。
- `bool` 只能参与逻辑运算。
- `bool` 本身只有 0/1 两种表示，实际上只需要 1bit 存储就可以了，为什么在 C++ 中也要将 `bool` 存储为 8bit 的值呢？
- `void` 型称为无值型。`void` 型是一种较为抽象的概念，在 C++ 语言中它用来说明函数及其参数，没有返回值的函数说明为 `void` 类型的函数，没有参数的函数其形参表由 `void` 表示。
- 有了 `void` 类型，C++ 语言规定，所有函数说明（`main` 函数除外）都必须指明（返回）类型，都必须包含参数说明。

目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架：主函数/预处理命令/SP/OOP框架
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

派生数据类型

- 基本类型经过简单的字长及范围放大或缩小，就形成了基本类型的简单派生类型。
- 派生类型的说明符由 `int`、`float`、`double`、`char` 前面加上类型修饰符组成。类型修饰符包括：
 - `short`: 短的，缩短字长。
 - `long`: 长的，加长字长。
 - `signed`: 有符号的，值的范围包括正负值。
 - `unsigned`: 无符号的，值的范围只包括正值。

派生数据类型

表 3.2 基本类型及派生类型的字长及值域

类 型 名	字长 (B)	取 值 范 围
bool		{false,true}
char	1	-128 ~ 127
(signed) char	1	-128 ~ 127
unsigned char	1	0 ~ 255
short (int)	2	-32 768 ~ 32 767
(signed) short (int)	2	-32 768 ~ 32 767
unsigned short (int)	2	0 ~ 65 535
int	4	-2 147 483 648 ~ 2 147 483 647
(signed) int	4	-2 147 483 648 ~ 2 147 483 647
unsigned (int)	4	0 ~ 4 294 967 295
long (int)	4	-2 147 483 648 ~ 2 147 483 647
(signed) long (int)	4	-2 147 483 648 ~ 2 147 483 647
unsigned long (int)	4	0 ~ 4 294 967 295
float	4	-3.4E38 ~ 3.4E38
double	8	-1.7E308 ~ 1.7E308
long double	10	-1.1E4932 ~ 1.1E4932
void	0	{ }

数据类型的内存量

Code

type_demo.cpp

```
1 // helloworld.cpp
2 #include "type_demo.h"
3
4 int main(void)
5 {
6     sizeoftype();
7     return 0;
8 }
```

type_demo.h

```
1 // helloworld.cpp
2 #include <iostream>
3 using namespace std;
4
5 void sizeoftype(){
6     cout<<"sizeof(int)="<<sizeof(int)<<endl;
7     cout<<"sizeof(float)="<<sizeof(float)<<endl;
8     cout<<"sizeof(double)="<<sizeof(double)<<endl;
9     cout<<"sizeof(wchar_t)="<<sizeof(wchar_t)<<endl;
10    cout<<"sizeof(char)="<<sizeof(char)<<endl;
11    cout<<"sizeof(bool)="<<sizeof(bool)<<endl;
12 }
```

Run

```
sizeof(int)=4
sizeof(float)=4
sizeof(double)=8
sizeof(wchar_t)=2
sizeof(char)=1
sizeof(bool)=1
```

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

枚举类型

- `enum` 类型又称枚举类型，它是一种由用户参与定义的类型。
- 声明格式为：

`enum` <**`enum`** 类型名>{<枚举值表>} <枚举变量表>;

- 示例：

```
enum color { RED= 1, YELLOW, BLUE } c1= BLUE, c2;
```

```
enum color  a, b = RED, c;
```

```
enum day { Sun,Mon,Tue,Wed,Thu,Fri,Sat };
```

```
enum day1 { Mon=1, Tue,Wed,Thu,Fri,Sat,Sun };
```

枚举类型

- `enum` 类型实际上是 `int` 型的一个子集，其每一个值对应一个整数。
- `n` 个枚举值全未赋常量值时，自左至右分别与整数 `0~n-1` 对应。
- 若第 `i` 个枚举值赋的常量值为整数 `m`，则其未赋常量值的后续枚举值分别与整数 `m+1`、`m+2`，... 对应，直到下一个赋了值的枚举值或结束。因此，为枚举值所赋的整型常量值应自左至右递增。
- 枚举类型变量只能赋予其值表中的值，且不能直接赋予数值。
- 枚举类型的说明亦可作为成组说明若干整型符号常量的方法。它不是完全由用户定义的。

```
enum color { RED= 1, YELLOW, BLUE } c1= BLUE, c2;
```

```
enum color  a, b = RED, c;
```

```
enum day { Sun, Mon, Tue, Wed, Thu, Fri, Sat };
```

```
enum day1 { Mon=1, Tue, Wed, Thu, Fri, Sat, Sun };
```


目录

- Hello World
 - 安装运行环境：vscode/docker/linux+g++/win+wsl2
 - 基本元素：注释/头文件/函数/语句/标准输入输出流
 - 基本符号与ASCII码
 - C++的词汇列表：关键字/标志符/字面常量/运算符/分隔符/
 - C++程序的基本框架：主函数/预处理命令/SP/OOP框架
 - 运行C++程序：编辑-预处理-编译-链接-调试
- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

类型说明：常量

- C++程序中的数据可分为**常量（constant）**与**变量（variable）**两大类。在程序执行过程中其值不能被改变的数据称为常量，其值可以改变的数据称为变量。常量又分**有名常量**和**字面常量**。
- **字面常量**的类型是根据书写形式来区分的。例如，475、200 是 int 型，3.1416、200.0 是float 型，'a'、'4'、'@'是 char 型。
- **有名常量**和**变量**在程序中必须遵循“**先声明，后使用**”的原则，程序中出现的所有有名常量和变量都必须在使用前由常量说明语句和变量说明语句进行说明。

```
const float pai = 3.1416;
```



类型说明：常量

- 常量说明语句的格式必须以**关键字 const** 开头为：

const <类型名><常量名> = <表达式>;

const float pai = 3.1416;

- 表达式的值应与该常量类型一致的表达式（常量和变量也是表达式）

`char* str = "Hello C++";`  `const char* str = "Hello C++";`

warning: ISO C++ forbids converting a string constant to 'char*'

- 常量和变量都要求系统为其分配内存单元，所以可以把有名常量视为一种**不允许赋值改变的或只读不写**的变量，称为 **const 变量**。
- 与宏定义的区别：宏定义也能定义常量，不过，用宏替换的方法定义符号常量与 **const** 方式的实现机制是不同的：
 - 宏替换是在编译时把程序中出现的所有标识符 **N** 或 **pai** 都用 **1000** 和 **3.1416** 来**替换**，这里并没有一个只读不写的 **const** 变量存在；
 - 宏替换的方式中没有类型、值的概念，仅是两个字符串的代换。

类型说明：变量

- 变量是数据在程序中出现的主要形式，在变量第 1 次被使用前它应被说明。变量说明的格式为：

[<存储类>] <类型名或类型定义><变量名表>;

```
static long sum;
```

- 类型名或类型定义在任何变量说明语句中都必须包含不可缺省。
- 变量名表可连续定义多个并赋初始值

变量名表：<变量名>[=<表达式>]，<变量名表>

```
int size, high, temp= 37;
```

类型说明：变重-全局变重和局部变量

- 全局变量：其说明语句不在任何一个类定义、函数定义和复合语句（程序块）之内的变量。全局变量所占用的内存空间在内存的数据区，在程序运行的整个过程中位置保持不变。
- 局部变量：其说明语句在某一类定义、函数定义或复合语句之内的变量。简单地说，说明语句包含在某一对分割符“{”和“}”之内的变量为局部变量，否则为全局变量。
- 局部变量占用的空间一般位于为程序运行时设置的临时工作区，以堆栈的形式允许反复占用和释放。

```
4  int global_a = 0;
5  int main(void)
6  {
7      int local_a = 1;
8      sizeof type();
9      cout<<"global_a="<<global_a<<" , local_a="<<local_a<<endl;
10     return 0;
11 }
```

类型说明：变量-生存期与作用域

- 生存期：变量 **a** 的生存期是指从 **a** 被说明且分配了内存开始，直到该说明语句失去效力，相应内存被释放为止。
- 作用域：变量 **a** 的作用域是指标识符 **a** 可以代表该变量的范围，一般与生存期一致，但由于 C++ 语言 **允许在不同程序部分为不同的变量取同一名字**，因此一个变量名的作用域可能小于其生存期。

```
4  int global_a = 0;
5  int main(void)
6  {
7      int local_a = 1;
8      {
9          float local_a = 2.0;
10         cout<<"global_a="<<global_a<<" , local_a="<<local_a<<endl;
11     }
12     sizeof type();
13     cout<<"global_a="<<global_a<<" , local_a="<<local_a<<endl;
14     return 0;
15 }
```

类型说明：变量-存储类属性

- **register**: 把变量说明为寄存器变量，将可能以寄存器作为存储空间。**register 仅能建议（而不是强制）**系统使用寄存器，这是因为寄存器个数有限，当寄存器不够用时，该变量仍按常规变量处理。
- **register**定义变量时**仅能定义为局部变量**。

```
7   register float cum = 0.0;
8   for(int i=0;i<1000;i++){
9       |   cum +=i;
10  }
```



```
5   register float cum = 0.0;
6   int main(void)
7   {
8       |   for(int i=0;i<1000;i++){
9           |       cum +=i;
10          }
```

类型说明： 变量-存储类属性

- **static**: 把变量说明为静态变量，任何静态变量的生存期将延续到整个程序的终止。这意味着：
 - 静态变量和全局变量一样，在内存数据区分配空间，在整个程序运行过程中不再释放。
 - 静态变量如未赋初值，系统将自动为其赋缺省初值 0（NULL）。
 - 静态变量的说明语句在程序执行过程中多次运行或多次被同样说明时，其第一次称为定义性说明，进行内存分配和赋初值操作，在以后重复说明时仅维持原状，不再做赋初值的操作。
 - **static** 变量是“永久”占用空间，因此可以保存函数调用过程中某些局部量的结果并把它传送到该函数的下次调用之中。由于静态变量容易浪费空间，所以不宜过多使用。

类型说明： 变量-存储类属性

Code

```
22 void acm_variable(){
23     float acm = 0.0;
24     acm+=1;
25     cout<<"acm_variable.acm = "<<acm<<endl;
26 }
27
28 void acm_static(){
29     static float acm = 0.0;
30     acm+=1;
31     cout<<"acm_static.acm = "<<acm<<endl;
32 }
12     for(int i=0;i<5;i++){
13         acm_variable();
14         acm_static();
15     }
```

Run

```
acm_variable.acm = 1
acm_static.acm = 1
acm_variable.acm = 1
acm_static.acm = 2
acm_variable.acm = 1
acm_static.acm = 3
acm_variable.acm = 1
acm_static.acm = 4
acm_variable.acm = 1
acm static.acm = 5
```

类型说明： 变量-存储类属性

- **extern**: 把变量说明为外部变量。
- 一个变量被说明为外部变量，其含义是告诉系统不必为其分配内存，该变量已在这一局部的外面定义。
- 外部变量一般用于由多个文件组成的程序中，有些变量在多个文件中被说明，但却是同一变量，指出某一变量为外部变量就避免了重复分配内存。
- 可以在程序中多次说明**extern**变量（声明），但是为**extern**变量赋初值会报警（定义），但是并不会出错，只是等价于**extern**失效。

```
4 // int global_a = 0;
5 int global_a=0;
6 extern int global_a;
7 extern int global_a;
```

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

命名空间

- 名字空间（namespace）亦称命名空间，是用来解决大型程序中标识符重名的问题。由标识符命名的变量、常量、函数、类、对象等可以根据它们的逻辑关系分组，每个组就是一个名字空间，说明一个名字空间的语法格式为：

namespace <标识符> { <若干说明或定义> }

```
8   namespace name1{
9       int i,j,k;
10  }
11  namespace name2{
12       int i,j,k;
13  }
14  int main(void)
15  {
16       name1::i = 1;
17       name1::j = 2;
18       name1::k = 3;
19       name2::i = 4;
20       name2::j = 5;
21       name2::k = 6;
```

命名空间

- 引用名字空间的变量时都要加上名字空间前缀，会比较麻烦，对此，C++语言又引入了使用指令：

using namespace <名字空间名>

- C++语言中的标准函数库的变量与函数都属于名字空间 `std`，如 `cin`、`cout` 等，应写成 `std::cin`、`std::cout`，不过，由于在相应的头文件中已包括使用指令：`using namespace std`，因此，前缀 `std::` 可以省略，若程序中又把 `cin` 说明为变量，则在相应的作用域 `cin` 代表该变量，而在输入语句中需加限定符：`std::cin>>...`。

```
C type_demo.h
1  // type_demo.h
2  #include <iostream>
3  using namespace std;
```

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

类型别名

- 用关键字 **typedef** 引导的类型说明语句，可由用户为一个已定义的类型赋予一个新的类型名（别名），其格式为：

typedef <已定义类型名> <新类型名>

```
16  typedef int studentid;  
17  int main(void)  
18  {  
19      studentid gongcheng = 1210517;  
20      cout<<"sizeof(studentid)="<<sizeof(studentid)<<endl;
```

gongcheng's id = 1210517, sizeof(studentid)=4

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

运算符和表达式的概念

- 运算符与表达式的概念，来源于数学，计算机却看不懂这些符号，让计算机完成一个表达式的计算，还需要编写一系列的指令。
- 表达式由运算分量和运算符按一定规则组成，其中的运算符指明所进行运算的类型，运算分量可以是常量、变量或表达式，单个的常量或变量也是一个表达式。
- 按参加某一类运算的运算分量的个数，运算分为单目运算、双目运算以及三目、多目运算。

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

运算与运算符类型-赋值运算

- 一类运算是一个运算类型，具有相同运算分量和结果类型的运算划分为同一类。比如**赋值运算**、**算术运算**、**关系运算**、**逻辑运算**、**位运算**等。
- **赋值运算**是一种双目运算，其形式为：<变量名>=<表达式>
 - “=”为赋值运算符（与数学中的等号含义不同）。
 - **左运算分量为与表达式类型相同的变量**。（左值和右值）
- 赋值运算的操作为：1,计算表达式的值；2,把该值赋给左端变量
- 除赋值运算符“=”之外，C++还提供了另外 10 种复合赋值运算符，它们是：
 $+=$ 、 $-=$ 、 $*=$ 、 $/=$ 、 $\%=$ 、 $>>=$ 、 $<<=$ 、 $\&=$ 、 $|=$ 、 $\wedge=$ ，
- 这些符号仅是一种简洁的表示方法。
- **注意赋值运算有返回值，返回值为左值。**

运算与运算符类型-赋值运算

Code

• signs_and_operators.cpp

```
1 // signs_and_operators.cpp
2 #include "signs_and_operators.h"
3 int main(void)
4 {
5     int a=1,b=2,c,d;
6     d=(c=a+b);
7     cout<<"c="<<c<<"", d="<<d<<endl;
8     return 0;
9 }
```

Run

```
$ g++ ./signs_and_operators.cpp -o signs_and_operators.exe
$ ./signs_and_operators.exe
c=3,d=3
```

运算与运算符类型-算术运算

- **算术运算**是指 `int` 型、`float` 型（也包括 `char` 型）的数值类数据计算后，得到**同一类型数据**的运算，包括：
 - 单目的减（-）、增量（++）和减量（--）运算。
 - 双目的加（+）、减（-）、乘（*）、除（/）和模（%）运算。
- **前缀增量**：`++<运算分量>`；**后缀增量**：`<运算分量>++`。运算分量为 `int` 型（包括 `char` 型）变量，具体操作为令变量值加 1。
- 前增量`++i` 与后增量 `i++` 的区别在于：如果`++i` 和 `i++` 作为表达式参加其他运算，则前者是先令 `i` 加 1 然后参加其他运算，而后者则是先令 `i` 参加其他运算，然后再令 `i` 加 1。

`i++;` // 结果等价于 `i = i + 1`

`++i;` // 结果等价于 `i = i + 1`

`a = i++;` // 结果等价于 { `a = i; i = i + 1;` }

`a = ++i;` // 结果等价于 { `i = i + 1; a = i ;` }

运算与运算符类型-算术运算

- 两个运算分量应为同一类型，如果不同，应该遵循类型转换原则，即由“短”类型向“长”类型的自动转换，即按照：
char→int→float→double 的次序进行自动转换。(保证不会溢出)

```
int a, b;
```

```
float x, y;
```

```
x = b * a + y;
```

- 两个 **int** 型数据相除，结果应为 **int** 型，若商不是整数，也要取整。
int 型与 **float** 或 **double** 型相除，结果应为 **float** 或 **double** 型。

```
11 void test_int_div(){  
12     int a = 3, b=2;  
13     float y = 2.0;  
14     cout<<"a/b="<<a/b<<" , a/y="<<a/y<<endl;  
15 }
```

```
$ g++ ./signs_and_operaters.cpp -o signs_and_operaters.exe
```

```
$ ./signs_and_operaters.exe
```

```
a/b=1, a/y=1.5
```

运算与运算符类型-算术运算

- 取模运算符%主要应用于整型数值计算。 $a\%b$ 表示用 b 除 a 所得到的余数。对于整数（`int` 型和 `char` 型）来说，除法运算和取模运算有如下关系：

$a - b * (a / b) == a \% b$ //这里“`==`”为等号

```
17 void test_int_mod(){
18     int a = 3, b=2;
19     cout<<"a - b *(a / b)="<<(a-b*(a / b))<<"\na % b="<<a % b<<endl;
20 }
```

```
$ g++ ./signs_and_operaters.cpp -o signs_and_operaters.exe
$ ./signs_and_operaters.exe
a - b *(a / b)=1
a % b=1
```

运算与运算符类型-关系运算

- 关系运算的是计算两个运算分量的值以后对其进行比较，若符合运算符指出的关系，其结果为 1（true），否则为 0（false）。
- C++提供 6 种关系运算，它们相应的运算符为：

<、<=、>、>=、==、!=

<运算分量><关系运算符><运算分量>

- 运算分量应该为数值类型（或称算术类型）和指针类型的表达式，运算符两边类型应该相同，**如果不同会启动隐式转换或报错。**
- 运算的结果类型为 **bool** 型，也就是逻辑值 false 和 true，这两个逻辑值对应于整数 0 和 1。
- C++语言中，bool型和 int 型都属于整数类型，bool值 false和 true 同时具有 int 型的值 0和 1，整数也可以**隐式地**转换为 bool值：非零整数转换为 **true**，而 0 转换为 **false**。

运算与运算符类型-关系运算

Code

```
22 void test_relation_op(){
23     bool a = (2>=2.5);
24     bool b = (int(2)>=int(2.5));
25     bool c = 7;
26     int d = true;
27     cout<<"a = "<<a
28         <<" , b = "<<b
29         <<" , c = "<<c
30         <<" , d = "<<d
31         <<endl;
32 }
```

Run

```
$ g++ signs_and_operaters.cpp -o signs_and_operaters.exe
$ ./signs_and_operaters.exe
a = 0, b = 1, c = 1, d = 1
```

运算与运算符类型-逻辑运算

- 逻辑运算也可以称为bool运算，只对0/1元素进行运算，也是计算机和硬件运算的基本原理，可以进行逻辑推理。
- C++提供 3 种逻辑运算符：！、&&、||，分别称为逻辑非、逻辑与、逻辑或运算。

<逻辑运算符!><运算分量>

<运算分量><逻辑运算符&&或||><运算分量>

- 运算分量应该是相同的数值（算术）类型或指针类型的表达式，其运算结果为 bool（int）型，且只能取 false（0）或 true（1）。

运算与运算符类型-逻辑运算

- 逻辑运算是按照真值表进行运算输出的，具体如下：
- 单目逻辑运算符！ 真值表：

分量	0	1
结果	1	0

- 双目逻辑与运算符&&真值表：

分量 1	0	1	0	1
分量 2	0	0	1	1
结果值	0	0	0	1

运算与运算符类型-逻辑运算

- 双目逻辑或运算符||真值表：

分量 1	0	1	0	1
分量 2	0	0	1	1
结果值	0	1	1	1

- 一般逻辑运算（以及关系运算==）的运算分量，**不取浮点类型**，因为在判定浮点数是否为0或两值是否相等时容易出错。

运算与运算符类型-逻辑运算

Code

```
41 void test_logic_op(){
42     int a = 2, b = 5;
43     cout <<"!a = "<< !a << ", !(b/a-a) = " << !(b / a - a)<< endl;
44     cout <<"a&&b = "<< (a && b) << ", (a&&(b/a-a)) = " << (a && (b / a - a)) << endl;
45     cout <<"a||b = "<< (a || b) << ", (a||(b/a-a)) = " << (a || (b / - a) )<< endl;
46 }
```

Run

```
!a = 0, !(b/a-a) = 1
a&&b = 1, (a&&(b/a-a)) = 0
a||b = 1, (a||(b/a-a)) = 1
```

运算与运算符类型-位运算

- 位运算是高级语言中的“低级”运算，其操作的对象是整型数，要对机器内部二进制表示的每一位（bit）进行运算。
- 双目位运算符：&、|、^、>>、<<
- 单目位运算符：~
- 它们的使用格式为：
 <运算分量> <双目位运算符&, | 或^> <运算分量>
 <运算分量> <双目位运算符>>或<<> <整数 n>
 <单目位运算符~> <运算分量>
- 运算分量为相同的 int 或 char 型及其派生类型，整数 n 为移位数，运算结果仍是整型。

运算与运算符类型-位运算

- 位运算是按位进行的逻辑运算，所以其也是根据真值表进行计算输出的，以char a = 39, b = 45为例，计算结果如下：

- 按位与运算&：

a & b的值是 37，计算过程按位进行逻辑与运算。

39	0	0	1	0	0	1	1	1
45	0	0	1	0	1	1	0	1
37	0	0	1	0	0	1	0	1

- 按位或运算|：

a | b的值是 47，计算过程按位进行逻辑或运算。

39	0	0	1	0	0	1	1	1
45	0	0	1	0	1	1	0	1
47	0	0	1	0	1	1	1	1

运算与运算符类型-位运算

- 位运算是按位进行的逻辑运算，所以其也是根据真值表进行计算输出的，以char a = 39, b = 45为例，计算结果如下：

- 按位异或运算 $\wedge$ ：

a $\wedge$ b 的值是 10，计算过程按位进行逻辑异或运算。

39	0	0	1	0	0	1	1	1
45	0	0	1	0	1	1	0	1
10	0	0	0	0	1	0	1	0

- 单目按位取反运算 $\sim$ ：

$\sim$ a 的值是 11011000。

- 按位左移运算 $\ll$ ：

a $\ll$ 1 等于 01001110，a $\ll$ 3 等于 00111000。

- 按位右移运算 $\gg$ ：

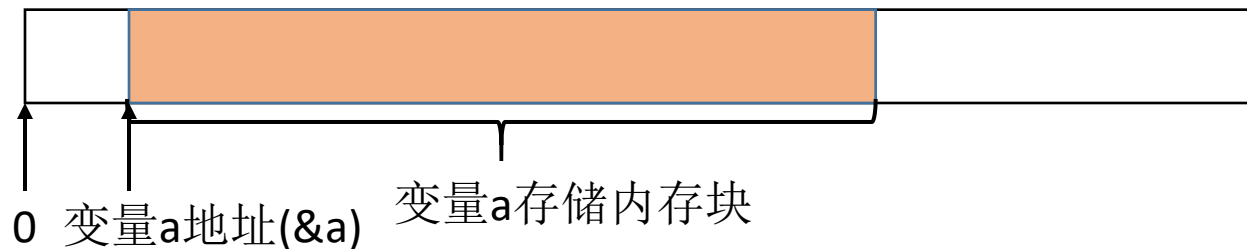
a $\gg$ 1 等于 00010011，a $\gg$ 3 等于 00000100。

运算与运算符类型-条件运算

- C++存在一个唯一的三目运算符：条件运算符，其符号表示为（?和:），其使用格式为：
$$\text{<表达式 1>?<表达式 2>:<表达式 3>}$$
- 其意义是：如果表达式1成立，则返回表达式2，否则返回表达式3。
- $s=(a>b)?a:b$ 等价于如下的语句：
$$\text{if (a > b) s = a; else s = b;}$$

运算与运算符类型-指针运算

- 指针运算符包括取地址运算符**&**和值引用运算符*****，它们都是单目运算符。
- 取地址运算为：**&<变量>**，返回变量的地址（指针）。
- 这里变量必须是一个**已经说明过的分配过内存的变量名**。
- 值引用运算为：***<指针变量>**，返回地址存储的值。指针变量是一个指针变量名。



运算与运算符类型-函数调用符

把圆括号()称为运算符并不符合人们的习惯，这是因为C++语言把函数调用和类型强制转换也归类为表达式。()作为运算符有两种情况：

- 用于函数调用，格式为：<函数名>(<实参表>)

例如：add(a, b)、main()等。

- 用于强制类型转换。其格式为：<类型名>(<表达式>) 或 (<类型名>) <表达式>

例如：

```
int a = 2, b = 3;
```

```
cout << a / b << " " << float(a)/ b << endl;
```

运算与运算符类型-字长提取符

字长提取符 `sizeof` 实际上是一个系统提供的函数。其格式为：

`sizeof` (<运算分量>)

这个单目运算和一般函数的使用有两点区别：

(1) 运算分量是一个变量名，也可以是一个类型名。

(2) 当运算分量是变量名时，括号 () 可以省略。运算的结果是一个整数，该整数表示指定变量或类型的长度，也就是字节数。

例如：

```
int a, b;
```

```
sizeof (char)      //返回值为 1
```

```
sizeof a           //返回值为 4
```

运算与运算符类型-动态分配符

`new` 和 `delete` 是 C++语言提供的用于动态数据生成和释放的单目运算符，其格式为：

`new` <类型名>

`new` <类型名>[**`size`**]

`new` <类型名> (<初值>)

`delete` <指针变量>

`delete` [] <指针变量>

(1) `new` 运算用来生成一个无名的动态变量，它返回一个该类型的指针值，在程序中利用指针对这个变量进行操作。

(2) `delete` 运算用来释放或撤销由 `new` 生成的动态变量。

运算与运算符类型-其他运算符

- 数组下标运算符：<数组名> [<下标表达式>]
- 限定运算符：<类名>::<类成员名>

<名字空间名> ::<变量名>

当名字空间名为空，即::a，表示a为全局变量。

- 成员选择符：

<对象名>.<成员(数据或函数)>

<对象指针>-><成员>

目录

- 数据类型
 - 基础数据类型
 - 派生数据类型
 - 枚举类型
 - 类型说明：常量/变量/静态/存储(auto/register/extern)
 - 命名空间
 - 类型别名
- 表达式与基础运算符
 - 运算符和表达式的概念
 - 运算与运算符类型
 - 运算符优先级

运算符优先级

- 在大多数情况下，程序中出现的表达式不止包含一个运算。在这样的一些表达式中，许多运算放在一起，**为了避免二义性**，必须有一个严格的规定，以保证每一种合法的表达式都只有唯一正确的计算顺序，这就是优先问题。
- C++语言中解决优先问题有 3 种方法：
 - **括号（）优先。**在表达式中可以对于其任意子表达式使用括号，括号内的部分优先计算。
 - **没有括号的地方，不同的运算按照优先级的顺序进行计算，优先级高的先计算。**
 - **具有相同优先级的运算符，按两种顺序规定决定先后次序：**
 - **左结合规则：**即从左向右依次运算，例如：
 - **右结合规则：**即从右向左依次运算

运算符优先级

表 3.3 运算符优先级列表

运 算	运 算 符	格 式	例 句	左/右结合
类限定	::	<类名>::<成员>	stack::pop()	
全局限定	::	::<全局变量>	::a	
成员选择	.	<对象>.<成员>	st1.push(a)	右
成员选择	—>	<对象指针>—><成员>	stp—>push(a)	右
下标	[]	<数组名>[<下标>]	array[i+1]	
函数调用	()	<函数名>(<实参表>)	sin(x)	
后增量	++	<变量>++	a++	右
后减量	--	<变量>--	a--	右
变量长度	sizeof	sizeof<变量>	sizeof(a)	
类型长度	sizeof	sizeof<类型>	sizeof(char)	
取地址	&	&<变量/对象>	&a	右
指针引用	*	*<指针>	*pa	右
单目减	-	-<变量>	-a	右

运算符优先级

运 算	运 算 符	格 式	例 句	左/右结合
前增量	++	++<变量>	++a	右
前减量	--	--<变量>	--a	右
逻辑非	!	!(<逻辑表达式>)	!(a>b)	右
按位取反	~	~(<整型表达式>)	~(i+3)	右
动态分配	new	new<类型>	new char	
动态撤销	delete	delete<指针>	delete pc	
数组撤销	delete	delete[]<指针>	delete[] pc	
乘法	*	<表达式>*<表达式>	a * b	左
除法	/	<表达式>/<表达式>	a / b	左
取模	%	<表达式>%<表达式>	a % b	左
加法	+	<表达式>+<表达式>	a + b	左
减法	-	<表达式>-<表达式>	a - b	左
左移	<<	<整型量><<<表达式>	a << 5	左
右移	>>	<整型量>>><表达式>	a >> i	左
小于	<	<表达式><<表达式>	a < b	左
不大于	<=	<表达式><=<表达式>	a <= b	左
大于	>	<表达式>><表达式>	a > b	左
不小于	>=	<表达式>>=<表达式>	a >= b	左

运算符优先级

等于	==	<表达式>==<表达式>	a == b	左
不等于	!=	<表达式>!=<表达式>	a != b	左
按位与	&	<整表达式>&<整表达式>	a & b	左
按位异或	^	<整表达式>^<整表达式>	a ^ b	左
按位或		<整表达式> <整表达式>	a b	左
逻辑与	&&	<表达式>&&<表达式>	(a>b)&& c	左
逻辑或		<表达式> <表达式>	(a>b) c	左
条件	? :	<表达式>?<表达式>:<表达式>	a>b?a:b	右
赋值	=	<变量> = <表达式>	a = a + b	右
乘赋值	*=	<变量> *= <表达式>	a *= b	右
除赋值	/=	<变量> /= <表达式>	a /= b	右
模赋值	%=	<变量> %= <表达式>	a %= b	右
加赋值	+=	<变量> += <表达式>	a += b	右
减赋值	-=	<变量> -= <表达式>	a -= b	右
左移赋值	<<=	<变量> <<= <表达式>	a <<= b	右
右移赋值	>>=	<变量> >>= <表达式>	a >>= b	右
与赋值	&=	<变量> &= <表达式>	a &= b	右
或赋值	=	<变量> = <表达式>	a = b	右
异或赋值	^=	<变量> ^= <表达式>	a ^= b	右
逗号	,	<表达式>,<表达式>	a = 3,a ++	左

运算符优先级

- 具有相同优先级的运算符，按两种顺序规定决定先后次序：
- 左结合规则：即从左向右依次运算，包括：双目的算术运算符、关系运算符、逻辑运算符、位运算符、逗号运算符。

$a + b - c$ 等价于 $(a + b) - c$

- 右结合规则：即从右向左依次运算，包括可以连续运算的单目运算符、赋值运算符、条件运算符。

$* \& \text{array}$ 等价于 $* (\&\text{array})$

$d = c = 4 * a * a$ 等价于 $d = (c = 4 * a * a)$

思考题

- 常量和变量的区别是什么，为什么要区分常量和变量？
- 在变量的声明和使用过程中，计算机都作了哪些工作？
- 全局变量与局部变量有什么不同，变量的作用域和生存期是否一致？
- 为什么要引入名字空间的概念，如何对它进行使用？
- “==”运算符和“=”运算符的使用含义是什么，与数学中的“=”运算符的使用含义相同吗？
- 前缀增量++i 与后缀增量 i++的使用区别是什么？试举一个实际例子来说明这种使用区别。
- 若参加运算的两个分量的类型不同时，系统将怎样来处理它们？
- 算术运算符、关系运算符和逻辑运算符的相对优先级关系是怎样的？

练习题

1. 设有如下的说明：

```
int i = 8, j = 3, k, a, b;
```

```
unsigned long w = 5;
```

```
double x = 1.42, y = 5.2, t, f;
```

试判断以下表达式是否正确，如果正确，试写出表达式的值以及表达式执行后其中相关联（被改变后）的各变量值。如“ $k = i++$ ”表达式的值为 8，相关联即被改变后的变量 k 的值为 8，而 i 值变为 9。

```
k = i++,
```

```
w += -2,
```

```
y += x++,
```

```
i /= j+12,
```

```
b = - -j,
```

```
a = 2*a = 3,
```

```
++ (i+j),
```

```
20/3+11%3,
```

```
20/3.0 + 'a' - 'B',
```

```
f = 3/2* (t=20.2-32.1),
```

```
k = (a=2, b=3, a+b)。
```

2. 已知 $\text{int } i = 32, j = 1, k = 3$ ，试确定下面的复合关系测试时产生真还是假。

```
!(!(true||false))
```

```
(true && false) && !(true||false)
```

```
i && j && k-3
```

```
34-i||k
```

```
j!=k && i!=k
```

```
!!!(i=6)
```

```
(5>3) && (3>1)
```

```
5>3 && 3>1
```

```
5>3>1
```

```
!i||(j-k) && i && !(k-3||i*k)
```